

QUESTION 5

Title

Longest Repeating Subsequence Using Dynamic Programming

Aim

To find the length of the longest repeating subsequence in a given string using Dynamic Programming and analyze its efficiency.

Problem Statement

Given a string, determine the length of the longest sub-sequence that appears more than once without using the same character index in both occurrences.

The sub-sequences must be identical but should originate from different positions in the string.

Sample Input

AABEBCDD

Expected Output

Length = 3

Introduction

A sub-sequence is a sequence that can be derived from a string by deleting some characters without changing the order of the remaining characters.

In many applications such as bio-informatics, text analysis, pattern matching, and data compression, finding repeated patterns is important.

The Longest Repeating Sub-sequence (LRS) problem aims to identify the longest sub-sequence that occurs at least twice in a string.

Dynamic Programming provides an efficient solution by storing intermediate results and avoiding repeated calculations.

Objective

- Find the longest repeating sub-sequence.
- Avoid using the same character index twice.
- Apply Dynamic Programming for efficient computation.
- Analyze algorithm performance.
- Determine the maximum repeating length.

Theory

The Longest Repeating Sub-sequence problem is similar to the Longest Common Sub-sequence (LCS) problem.

The difference is that the string is compared with itself.

A character match is considered only when:

$$s[i] = s[j]$$

and

$$i \neq j$$

This ensures that the same character position is not reused.

Dynamic Programming Approach

Let:

$$DP[i][j]$$

represent the length of the longest repeating sub-sequence considering the first i and j characters of the string.

Recurrence Relation

If:

$s[i-1] = s[j-1]$

and

$i \neq j$

Then:

$DP[i][j] = 1 + DP[i-1][j-1]$

Otherwise:

$DP[i][j] = \max(DP[i-1][j], DP[i][j-1])$

Algorithm

Step 1

Read the input string.

Step 2

Create a Dynamic Programming table of size $(n+1) \times (n+1)$.

Step 3

Compare every character with every other character.

Step 4

If characters match and indices are different, increase the subsequence length.

Step 5

Otherwise, store the maximum value from neighboring cells.

Step 6

Return the value stored in $DP[n][n]$.

Pseudocode

LRS(s)

$n \leftarrow \text{length of string}$

Create $DP[n+1][n+1]$

```

FOR i ← 1 TO n
  FOR j ← 1 TO n
    IF s[i-1] = s[j-1] AND i ≠ j
      DP[i][j] ← 1 + DP[i-1][j-1]
    ELSE
      DP[i][j] ← max(DP[i-1][j], DP[i][j-1])
RETURN DP[n][n]

```

Source Code

```

def longest_repeating_sub-sequence(s):
    n = len(s)
    dp = [[0 for j in range(n+1)] for i in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, n+1):
            if s[i-1] == s[j-1] and i != j:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][n]

s = input("Enter String: ")
print("Length =", longest_repeating_subsequence(s))

```

Sample Input

AABEBCDD

Sample Output

Length = 3

Working Example

Input String:

AABEBCDD

Longest Repeating Sub-sequence:

ABD

Length:

3

Analysis**Why Dynamic Programming?**

- Avoids repeated calculations.
- Stores intermediate results.
- Provides efficient execution.
- Suitable for large input strings.

Advantages

- Optimal solution.
- Efficient memory usage.
- Faster than brute-force methods.
- Easy to implement.

Time Complexity

$O(n^2)$

The DP table contains $n \times n$ entries.

Space Complexity

$O(n^2)$

Additional space is required for the DP table.

Applications

- DNA Sequence Analysis
- Pattern Recognition
- Text Mining
- Data Compression
- Bioinformatics
- Natural Language Processing

Result

The Longest Repeating Sub-sequence was successfully identified using Dynamic Programming. For the input string "AABEBCDD", the longest repeating subsequence has a length of 3. The Dynamic Programming approach efficiently solves the problem with $O(n^2)$ time complexity and $O(n^2)$ space complexity.